



Building a UI for Robotics Using Flutter and Zenoh

EF IoT Webinar

Hugo Alberto Garcia - Bluecorn

June 17, 2026

Problem Statement

The data already flows — so why does the UI still need a server?



The UI Is Always Behind a Gateway

Every IoT fleet, edge deployment, or robot already shares a **data fabric** — devices exchange telemetry *and* commands directly, with delivery and QoS guarantees, over MQTT, DDS, Zenoh, or similar.

Then someone wants a UI — and a UI runs both ways:

- **Observe** — read live state: joint angles, a sensor fleet's readings, a camera feed
- **Act** — write back: drive the arm to a pose or **stop** it; recalibrate a sensor; set a valve or thermostat; clear an alarm

Yet the dashboard almost never *joins* that fabric — it reaches in through a **gateway** on **both** legs (a WebSocket bridge, `rosbridge`, a REST shim): a *whole extra tier*, and a **client/server chokepoint** that flattens QoS — telemetry *and* commands both go second-hand, right where control needs them most.

The data already flows — both ways — so why does the UI still need a server?

In IoT this is the reflex: many protocols converge and get **processed** by a server tier. And the server **earns its place** —

- **Cloud services** — data-mining, analytics, long-term storage
- **Management** — fleet orchestration, identity, access control
- **Aggregation** — many protocols and many sites funnelled into one

We are not advocating a serverless architecture. We're advocating that the server is **not the only solution** — and that letting the **UI, and the application behind it, live on the same fabric** as a first-class peer has real merits:

no extra tier to build and secure · no QoS flattened on the way through · direct two-way interaction

That IoT fabric is **already peers** — sensors, edge services, even zenoh-pico microcontrollers. To put the **UI** on it as *one more peer* — no bridge tier bolted on — you need a protocol that speaks the links IoT actually runs on: **TCP, UDP, QUIC, TLS, WebSocket**, even **serial** to the most constrained devices — peer-to-peer *and* brokered, with QoS end to end.

That protocol is **Zenoh** — and in IoT, this works **today**.

The theme: a robotics UI that's a **first-class peer on the robot's own fabric** — no gateway, no bridge tier. We just saw the pattern work in IoT; **robotics** is the part you *couldn't* do until recently.

To show *how* — and prove it on a real arm — we need a little shared vocabulary first:

- **What ROS is** — the framework every robot runs
- **What Zenoh is** — the data-centric protocol
- **How Zenoh becomes a robot's middleware** — and what it unlocks

Then the payoff: the **Dart binding**, the **app**, and a **real PincherX-100** moving over Zenoh.

Background: ROS and Zenoh

a little shared vocabulary — nodes, paradigms, key expressions

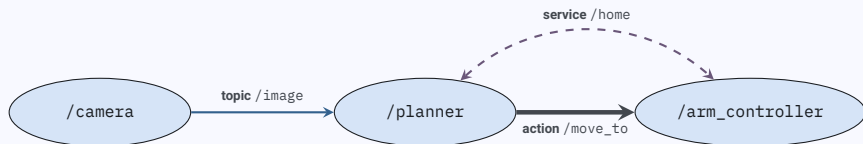


ROS — the Robot Operating System. Not actually an OS, but the **de-facto open-source framework** for building robot software (since 2007; **ROS 2** is the production-grade rewrite).

- **A robot's software is a graph of *nodes*** — small independent processes (a camera driver, a planner, a motor controller) running concurrently
- **They talk over three patterns: *topics*** (pub/sub streams), ***services*** (request/reply), and ***actions*** (long-running goals)
- **A vast ecosystem** — perception, navigation, manipulation — plus tooling: RViz (3D view), `rqt`, Gazebo (sim), the `ros2` CLI

The ROS Computation Graph

Those nodes form a **graph**, wired by the three patterns – **topics** (pub/sub streams), **services** (request/reply), and **actions** (long-running goals). This is the view `rqt_graph` renders:



Topics stream continuously (one $\rightarrow$ many) **services** answer on demand **actions** run long goals with feedback

Model: the ROS 2 computation graph (docs.ros.org; the view `rqt_graph` renders).

What Is Zenoh? — One Protocol, Three Paradigms

Eclipse Zenoh (/zeno/ — *Zero Overhead Pub/Sub, Store/Query & Compute*) is a **data-centric protocol**: one wire that unifies **data in motion** and **data at rest**. Open-source, written in Rust, with bindings in many languages.

Three communication paradigms over one data model:

Pub/Sub Publishers push; subscribers receive matching samples asynchronously (PUT & DELETE) — streams of telemetry and commands.

Get/Queryable Request/reply — the reply carries a value or an error.
REST-like verbs (put / get / delete; a payload-carrying get $\approx$ POST), but **name-based** — so a get is a **distributed query**, not a call to one host.

Storage Geo-distributed persistence — router plugins back data to RocksDB, InfluxDB, S3, or the filesystem, queryable via the *same* keys.

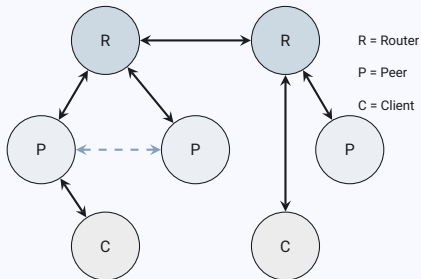
One protocol for all three — not three systems bolted together.

Three node roles:

Router zenohd daemon.
Brokers traffic,
bridges networks,
hosts plugins and
storage backends.

Peer Full-featured node.
Discovers other
peers, can relay
data in a mesh.

Client Lightweight
endpoint.
Connects to a
router or peer — no
relay capability.



Modes mix freely: peers mesh locally, clients connect to routers for WAN, routers bridge networks — localhost = LAN = WAN, one API.

The Data Model — Key Expressions

All data is addressed by **key expressions** — hierarchical, slash-separated paths with wildcard matching. These are the actual keys of today's demo robot:

Pattern	Example	Matches
Exact	px100/cmd/pose	that key only
Single *	px100/cmd/*	pose, jog, ...
Glob **	px100/**	entire subtree

- Same key space for pub/sub, queries, and storage
- A subscriber on px100/** receives publishes, query replies, *and* storage updates
- Interest-based routing — data only flows toward declared subscribers (no flooding)
- Key expressions can be *declared* on a session for wire-level optimization (integer ID instead of string)

Zenoh and Robotics

`rmw_zenoh` – the robot's middleware *is* Zenoh



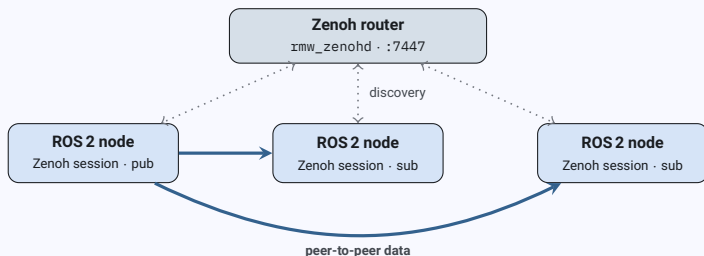
A robot runs **ROS 2** — a graph of nodes (drivers, planners, controllers). ROS 2 ships **no wire of its own**; it delegates transport to a **pluggable middleware** (the RMW). Historically DDS — and since 2023, **Zenoh**.

- **Official:** OSRF selected Zenoh as the alternative ROS 2 middleware; `rmw_zenoh` ships from **Jazzy (2024)** and is **Tier-1 since Kilted (2025)** — co-developed by Intrinsic/Google and ZettaScale
- **So the robot is a Zenoh network:** every node already publishes and subscribes over Zenoh; a router runs alongside (our robot: `:7447`, published to the LAN)

Every node on the same wire. What that unlocks for the *UI* is the payoff ahead.

How `rmw_zenoh` Provides Zenoh in the Robot

Each ROS 2 node *is* a **Zenoh session**; one **router** (`rmw_zenohd`, :7447) handles **discovery only** – then data flows **peer-to-peer**.



- **Discovery** = Zenoh **liveliness tokens** + the router's **gossip scouting** – *no UDP multicast* (the DDS pain point, designed out)
- **The router is never in the data path** – once peers find each other, messages go **node-to-node** on keys `<domain>/<topic>/<type>/<hash>`

Source: official `ros2/rmw_zenoh` design doc (`docs/design.md`).

You've seen it: with `rmw_zenoh`, the robot's *own* wire is Zenoh, discovered without multicast. So here is what it **means** —

Zenoh lets the app interact with the robot — through ROS — as a peer, eliminating the broker/gateway and the client/server tier every other approach forces in.

Newly possible. Before `rmw_zenoh`, a UI could only reach the robot through a bridge — now it joins the fabric itself.

The Binding and the App

the missing Dart binding, filled — and the app it builds



ROS2 officially supports two client languages — **C++** (rc1cpp) and **Python** (rc1py). A UI can be a node directly: **rqt**, the standard ROS GUI, is a **Qt + Python** node; Qt / C++ works the same way.

And **Zenoh widens the door**. It already has bindings for **Rust, C++, Python, Kotlin / Java, TypeScript** — each with its own UI toolkits. With `rmw_zenoh`, any of them could now join the robot's fabric (Jetpack Compose on Android, a TypeScript web app, ...).

This talk is FLUTTER — one codebase across mobile, desktop, and web, native widgets. But **Dart was the one language with no Zenoh binding**. So I wrote it.

Dart was the gap — so Flutter UIs, Dart services, and CLIs all sat **behind bridges**.

Filled: `package:zenoh` — pure Dart over `dart:ffi`, wrapping `zenoh-c`.
One binding opens the **whole Dart ecosystem**:

- **Flutter** — multi-platform operator UIs (this talk)
- **Server-side Dart** — Serverpod, Dart Frog, Shelf
- **CLI & tooling** — our 26 `z_*` examples are CLIs

`package:zenoh v0.19.0` today: 156 C functions, 571 integration tests, prebuilt for Linux x86_64 and Android.

github.com/bluecorn/zenoh_dart

The binding is plain Dart — declarations are **synchronous**; data arrives as a Stream. The canonical pub/sub “hello world,” two tiny programs sharing one key space:

Publisher

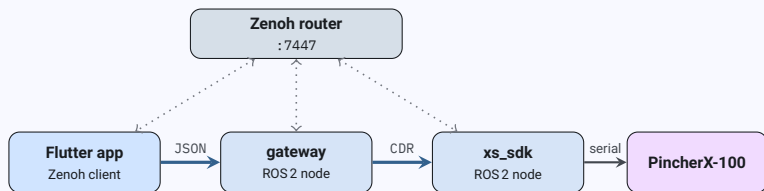
```
final session = Session.open();           // a peer on the fabric
session.put('demo/greeting', 'Hello, Zenoh!');
session.close();
```

Subscriber

```
final session = Session.open();
final sub = session.declareSubscriber('demo/**');
sub.stream.listen((s) => print('${s.keyExpr}: ${s.payload}'));
```

No broker to run, no bridge to cross — put here, the Stream fires there. Get/queryable, liveliness, and SHM follow the *same* shape.

The canonical way a UI connects to ROS is a **gateway** — a translation node between the app and the robot's graph. Our robot, a **PincherX-100**, runs one — a thin `rc1cpp` (C++) ROS2 node on `rmw_zenoh`. Because that middleware is Zenoh, every participant — the app included — is a **peer on one router**:

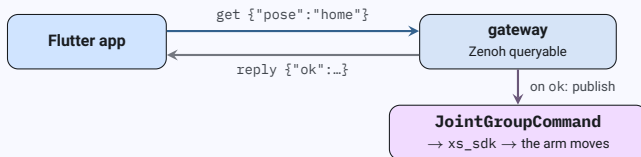


Each box is a **Zenoh session** on *one* router (**dotted** = discovery only). The app speaks **JSON** to the gateway; the gateway translates to the ROS **CDR** wire — **solid** = peer-to-peer data, the router never in the path. A **gateway node** still translates here; **the proof ahead removes even it**.

github.com/bluecorn/flutter_zenoh_gateway

Command & Ack — a Query, Not a Put

Zenoh ships **request/reply** as a built-in primitive (get & a queryable) — so a command carries its **own ack**, no second channel. A pose command is a query: the app gets px100/cmd/pose with the JSON; the gateway (a **queryable**) **always** replies.



Three outcomes — the app always learns which:

- **delivered** — `{"ok": true}`: the arm moves
- **rejected** — `{"ok": false}`: bad/unknown pose, no publish
- **error** — no reply in 3 s: gateway unreachable

A rejection is a **valid reply**, never a transport error (always `reply()`, never `reply_err`) — the app tells “*said no*” from “*didn’t answer.*” And the gateway is a **peer that earns its place**, not a forced broker — it owns the semantics.

The App's Entire Zenoh Boundary

```
/// The single `package:zenoh` boundary for the app.
class ZenohService {
  Session? _session;

  /// Open a CLIENT session to the robot's router (Session.open is sync).
  void connect(String endpoint) {
    final config = Config()
      ..insertJson5('mode', '"client"')
      ..insertJson5('connect/endpoints', '["$endpoint"]');
    _session = Session.open(config: config);
  }

  /// Send [payload] as a query on [key]; return the reply (ack) bytes.
  Future<Uint8List> query(String key, Uint8List payload) async {
    final reply = await _session!
      .get(key, payload: ZBytes.fromUint8List(payload),
        timeout: const Duration(seconds: 3))
      .first;
    if (!reply.isOk) throw StateError('transport error');
    return reply.ok.payloadBytes;
  }
}
```

The app's **complete** zenoh footprint, in **Dart** — connect + one query; the reply decodes to a **delivered / rejected / error** ack.

Two Ways In: A Bridge Tier, or a Peer on the Fabric

A Flutter app has two real routes to a ROS robot. **Both need a robot-side process – neither is “no server.”** The difference is where the app sits.

Flutter + rosbridge

- App → **WebSocket** → `rosbridge_server` → the ROS graph
- **Generic** access to *any* ROS topic / service / action, zero custom server code – its real strength
- Speaks the rosbridge protocol (JSON; CBOR for subscriptions)

Flutter + Zenoh (ours)

- App → **Zenoh** → the *same router the ROS nodes use* (via `rmw_zenoh`)
 - A **Zenoh client of the robot's own router**, on the same wire as the ROS nodes
 - A per-robot contract via our gateway – custom code, app-decoupled
-
- **Same, not magic:** both run a robot-side node; on an `rmw_zenoh` stack **both even ride Zenoh internally** (`rosbridge_server` is RMW-agnostic). The contrast is the **client leg** – a WebSocket island vs the same fabric.
 - **The difference that matters:** a bridge client **asks** a proxy for the ROS graph; a Zenoh client can **participate** – read `@ros2_liv` liveliness the way `rmw_zenoh` itself does. *Different, not better.*

Proof

the UI on the robot's fabric, for real



Demo

Mobile device → Zenoh → gateway → arm

(recording plays here – watch the endpoint, the connect, and the arm)

We Went Further: the UI on the Fabric Directly

Past the gateway demo — a spike where the Flutter/Dart app **joins the robot's own `rmw_zenoh` wire**, no gateway node. **Both halves ran on the real PincherX-100.**

Read — telemetry off the fabric

- Subscribes to the robot's own `joint_states`; **decodes the ROS CDR in Dart**
- Computes px100 forward-kinematics → a live gripper (x, y, z) readout
- **No gateway in the state path**
- FK **validated** vs `tf2_echo` on the sim (< 0.5 mm); the same math ran on the real arm's telemetry

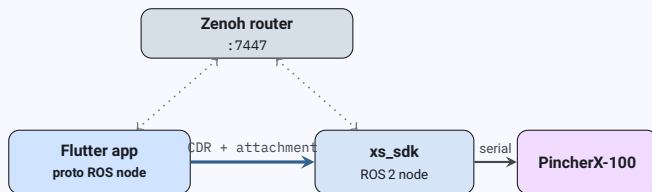
Command — the real arm moved

- Publishes `JointGroupCommand` on the `rmw_zenoh` wire (CDR + attachment); **the physical arm moved home ↔ sleep — no gateway, no token to actuate**
- Attachment is **load-bearing**, minted with the binding's `ZSerializer` (the `same ze_serializer_*` API `rmw_zenoh` uses) — correct **by construction**
- Negative control: a no-attachment publish was **silently dropped**

“No gateway **node**” — the Zenoh **router stays in the path** (as for *all* `rmw_zenoh` traffic); that is exactly what makes the app a **first-class Zenoh peer**.

The Same Router – No Gateway Node

In the spike, the **gateway is gone**: the Flutter app speaks the robot's `rmw_zenoh` wire *itself* – a **proto ROS node** on the same router.



Same router, same discovery (**dotted**) – the app now publishes **CDR + the mandatory attachment** *directly* to `xs_sdk` (**solid**, peer-to-peer). **No gateway in the path** – a **proto ROS node** actuating the real arm. The **command's shape changes too**: the gateway's **query/reply ack** becomes a **direct put** – still peers, trading that reply for fire-and-forget + telemetry feedback.

github.com/bluecorn/flutter_zenoh_direct

Conclusions

the pattern, the proof — and where it leads



**Not a cross-platform UI toolkit — a pattern:
the UI as a first-class peer on the device fabric.**

- **The problem** — the UI is the one component that never joins the data fabric; the usual fix is a gateway that costs a server and flattens your QoS
- **The pattern** — make the UI a **first-class Zenoh peer**; with `rmw_zenoh`, the robot already speaks it — no bridge tier in between
- **The binding** — `package:zenoh 0.19.0` opens the **whole Dart ecosystem** (Flutter, server-side, CLI) to Zenoh natively
- **The proof** — a Flutter app moved a real PincherX-100 over WiFi (gateway path, demoed); a spike then **read and commanded the arm directly** on the same fabric, **no gateway node**

The direct path — a **proto ROS node** on the robot's fabric — is the proven foundation the rest grows onto:

- **Graph citizenship** — announce via `@ros2_lv` so `ros2 node info / rqt` list the app: from a *proto* ROS node to a **graph-visible** one
- **The full app** — jog joints, subscribe to live state, record & play back poses (the teach-pendant dream)
- **Beyond the arm** — the same pattern spans cloud services, IoT sensors, and zenoh-pico MCUs; *any* language with Zenoh bindings can be a proto ROS node

Honest scope: **human-rate poses are proven**; high-rate jog/teach (needs put congestion control), graph citizenship, and general non-px100 messages are still research.

package: zenoh is **open source** (Apache-2.0), built on zenoh-c — and the whole stack develops with **zero hardware** (Docker + sim). Use it, file issues, contribute.



Thank You

github.com/bluecorn